```
# Time Series Forecasting of Foreign Exchange Rates
## EUR/USD and GBP/USD: Classical and Deep Learning Approaches

---

## Table of Contents

1. [Introduction](#1-introduction)
2. [Data and Methodology](#2-data-and-methodology)
3. [Classical Models](#3-classical-models)
4. [Advanced Models](#4-advanced-models)
5. [Results and Discussion](#5-results-and-discussion)
6. [Conclusion](#6-conclusion)
7. [References](#7-references)
8. [Appendix A: LLM Interaction](#appendix-a-llm-interaction)
9. [Appendix B: Code Documentation](#appendix-b-code-documentation)

---

## 1. Introduction

### 1.1 Motivation
Foreign exchange (FX) rate forecasting is a critical task in financial markets,
with applications in risk management, portfolio optimization, and trading
strategies. This project focuses on forecasting EUR/USD and GBP/USD exchange
rates using both classical time series methods and modern deep learning
approaches.

### 1.2 Objectives
- Implement and compare classical time series models (ARIMA, VAR, VECM, GARCH)
- Develop advanced deep learning models (LSTM, GRU, hybrid approaches)
- Evaluate model performance using appropriate metrics
- Validate results across both R and Python implementations

### 1.3 Research Questions
1. Which classical model provides the most accurate short-term forecasts for
EUR/USD?
2. Do deep learning models (LSTM, GRU) outperform classical methods?
3. Does a hybrid ARIMA+LSTM approach improve forecast accuracy?
4. Are the results consistent between R and Python implementations?

---

## 2. Data and Methodology
```

```
### 2.1 Data Description
- **Source**: Yahoo Finance
- **Assets**: EUR/USD and GBP/USD FX pairs
- **Frequency**: Daily
- **Period**: January 1, 2015 to present (≥5 years)
- **Target Variable**: Log returns of EUR/USD

### 2.2 Data Preprocessing
- Log returns calculation: $r_t = \log(P_t) - \log(P_{t-1})$
- Moving averages (20-day window)
- Realized volatility (20-day rolling standard deviation × √252)
- Train/test split: 90% training, 10% testing (chronological)

### 2.3 Evaluation Metrics
- **MSE** (Mean Squared Error)
- **RMSE** (Root Mean Squared Error)
- **MAE** (Mean Absolute Error)
- **MAPE** (Mean Absolute Percentage Error)

### 2.4 Statistical Tests
- **ADF Test**: Augmented Dickey-Fuller test for stationarity
- **KPSS Test**: Kwiatkowski-Phillips-Schmidt-Shin test for stationarity
- **Ljung-Box Test**: Autocorrelation in residuals
- **Jarque-Bera Test**: Normality of residuals
- **ARCH-LM Test**: Heteroskedasticity in residuals
- **Granger Causality**: Directional relationships between series
- **Johansen Test**: Cointegration rank determination
- **Diebold-Mariano Test**: Statistical comparison of forecast accuracy

---

## 3. Classical Models

### 3.1 ARIMA Models

#### 3.1.1 Model Specification
The AutoRegressive Integrated Moving Average (ARIMA) model is specified as:
$$\phi(B)(1-B)^d X_t = \theta(B) \epsilon_t$$

where:
- $\phi(B)$ is the autoregressive polynomial
- $\theta(B)$ is the moving average polynomial
- $d$ is the differencing order
- $\epsilon_t$ is white noise
```

3.1.2 Implementation
- **Manual ARIMA(1,0,1)**: Specified based on initial ACF/PACF analysis
- **Auto ARIMA**: Automatically selected using AIC criterion (max p=5, q=5)

3.1.3 Results
[Insert results table with model parameters, residual diagnostics, and forecast metrics]

3.2 VAR Model

3.2.1 Model Specification
Vector Autoregression (VAR) models the joint dynamics of multiple time series:
$$Y_t = c + A_1 Y_{t-1} + A_2 Y_{t-2} + \dots + A_p Y_{t-p} + \epsilon_t$$

where Y_t is a vector containing EUR/USD and GBP/USD log returns.

3.2.2 Implementation
- Lag order selected using AIC criterion (max lag = 15)
- Granger causality tests to assess directional relationships

3.2.3 Results
[Insert VAR lag selection results, Granger causality test results, and forecast metrics]

3.3 VECM Model

3.3.1 Model Specification
Vector Error Correction Model (VECM) accounts for cointegration between non-stationary series:
$$\Delta Y_t = \alpha \beta' Y_{t-1} + \Gamma_1 \Delta Y_{t-1} + \dots + \Gamma_{k-1} \Delta Y_{t-k+1} + \epsilon_t$$

where β contains the cointegrating vectors and α the adjustment coefficients.

3.3.2 Implementation
- Johansen cointegration test to determine cointegration rank
- VECM fitted with rank determined by trace statistic

3.3.3 Results
[Insert Johansen test results, cointegration rank, and forecast metrics]

3.4 GARCH Model

3.4.1 Model Specification

```
Generalized Autoregressive Conditional Heteroskedasticity (GARCH) models
volatility clustering:
$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

#### 3.4.2 Implementation
- GARCH(1,1) with zero mean
- Conditional volatility used for risk assessment

#### 3.4.3 Results
[Insert GARCH parameter estimates, conditional volatility plot, and forecast
metrics]

---

## 4. Advanced Models

### 4.1 LSTM Model

#### 4.1.1 Model Architecture
Long Short-Term Memory (LSTM) networks are a type of recurrent neural network
designed to capture long-term dependencies:
- Input layer: 30-day window of scaled returns
- LSTM layer: 32 units
- Dropout layer: 0.1
- Dense layers: 16 units (ReLU) → 1 unit (linear)
- Optimizer: Adam
- Loss: Mean Squared Error

#### 4.1.2 Training
- Epochs: 25
- Batch size: 32
- Validation split: 10%
- Time series cross-validation: 3-fold expanding window

#### 4.1.3 Results
[Insert training curves, CV RMSE, and forecast metrics]

### 4.2 GRU Model

#### 4.2.1 Model Architecture
Gated Recurrent Unit (GRU) is a simplified variant of LSTM:
- Same architecture as LSTM with GRU cells instead of LSTM cells
- Fewer parameters, potentially faster training

#### 4.2.2 Training
```

- Same hyperparameters as LSTM for fair comparison

4.2.3 Results
[Insert training curves, CV RMSE, and forecast metrics]

4.3 Hybrid ARIMA + LSTM

4.3.1 Model Specification
The hybrid approach combines classical and deep learning:
1. Fit ARIMA(1,0,1) to training data
2. Extract in-sample residuals
3. Train LSTM on residuals
4. Forecast: ARIMA forecast + LSTM residual forecast

4.3.2 Rationale
- ARIMA captures linear patterns
- LSTM captures non-linear patterns in residuals
- Combines strengths of both approaches

4.3.3 Results
[Insert forecast metrics and comparison with individual models]

---

5. Results and Discussion

5.1 Model Performance Comparison

[Insert comprehensive table comparing all models on MSE, RMSE, MAE, MAPE]

Model	MSE	RMSE	MAE	MAPE
ARIMA(1,0,1)	...	...	...	...
Auto ARIMA	...	...	...	...
VAR	...	...	...	...
VECM	...	...	...	...
GARCH	...	...	...	...
LSTM	...	...	...	...
GRU	...	...	...	...
Hybrid	...	...	...	...

5.2 Statistical Significance Testing

5.2.1 Diebold-Mariano Test
- H0: Two forecasts have equal accuracy

- Test compared: Auto ARIMA vs LSTM
- Result: [Insert statistic and p-value]
- Interpretation: [Accept/reject H0]

5.3 Residual Diagnostics

[Insert residual diagnostic results for all models]

5.4 Cross-Language Validation

Comparison of R and Python implementations for classical models:
- ARIMA parameters: [Compare]
- VAR lag selection: [Compare]
- Forecast metrics: [Compare]

5.5 Discussion

5.5.1 Key Findings
1. [Finding 1]
2. [Finding 2]
3. [Finding 3]

5.5.2 Model Recommendations
- For short-term forecasting: [Best model]
- For volatility modeling: [Best model]
- For interpretable results: [Best model]

5.5.3 Limitations
- [Limitation 1]
- [Limitation 2]
- [Limitation 3]

5.5.4 Future Work
- [Future direction 1]
- [Future direction 2]
- [Future direction 3]

---

6. Conclusion

This project implemented and compared classical and deep learning time series forecasting models for EUR/USD FX rates. Key conclusions:

1. [Conclusion 1]

2. [Conclusion 2]
3. [Conclusion 3]

The hybrid ARIMA+LSTM approach showed promising results, combining the interpretability of classical methods with the pattern recognition capabilities of deep learning. The R and Python implementations produced consistent results, validating the methodology.

---

7. References

[Insert references for:]
- ARIMA methodology (Box & Jenkins, 1970)
- VAR/VECM models (Lütkepohl, 2005)
- GARCH models (Bollerslev, 1986)
- LSTM networks (Hochreiter & Schmidhuber, 1997)
- Diebold-Mariano test (Diebold & Mariano, 1995)
- R packages: forecast, vars, urca, rugarch
- Python packages: statsmodels, pmdarima, arch, tensorflow

Appendix A: LLM Interaction

This appendix documents the interaction with Claude (Anthropic) during the project development.

A.1 Initial Setup

User Prompt:
> Main theme (required — same grading for both)
> Option A — Finance (Yahoo Finance: stocks, indices, FX, crypto; 2–3 correlated assets, daily, ≥5 years)
> Where should we put the advanced part (GARCH, LSTM/GRU)? Section 3 classical core must exist in BOTH R and Python.
> Advanced modeling mainly in Python (Jupyter notebook as main executable)
> Optional deepening (Block D, +2 pts) — what do you want included?
> Hybrid: ARIMA/Prophet + LSTM on residuals
> Generative AI section (4 pts): which LLM should we document in the report appendix?
> Claude (this chat — prompts/responses saved in appendix)
> Which 2–3 daily series (≥5 years) should we download from Yahoo Finance?
> EUR/USD + GBP/USD (FX pair)
> Primary series to forecast (others as multivariate / exogenous context)?

> Log returns of the first asset in the basket
> continue the mini project, if there is any other choice ask me

Claude Response:
> Based on my review, you have a solid Python implementation with classical models (ARIMA, VAR, VECM, GARCH) and deep learning models (LSTM, GRU, hybrid). However, I notice several things are still needed:
>
> **Missing components:**
> 1. **R implementation** - Section 3 classical core must exist in BOTH R and Python. You currently only have Python.
> 2. **Jupyter notebook** - The advanced modeling should be in a Jupyter notebook as the main executable (currently you have `run_pipeline.py`)
> 3. **Report/documentation** - Structure for the project report
> 4. **LLM documentation** - Appendix section for this conversation with Claude (prompts/responses)

A.2 Implementation Decisions

User Response:
> work depends on the steps of the delivrables

Claude Response:
> Based on what you've mentioned, I'll work on this in logical order:
>
> 1. **R implementation** (Section 3 classical core must exist in BOTH R and Python)
> 2. **Jupyter notebook** (advanced modeling in Python)
> 3. **Report structure**
> 4. **LLM documentation** (appendix)

A.3 Implementation Details

[Add subsequent prompts and responses as the project progresses]

Appendix B: Code Documentation

B.1 Project Structure

```
proj/
├── data/
│   └── fx_panel.csv              # Cached FX data
```

```
├── outputs/
│   ├── figures/                  # Diagnostic plots
│   │   ├── 01_panel.png
│   │   ├── acf_pacf_*.png
│   │   ├── stl_*.png
│   │   ├── garch_volatility.png
│   │   ├── rnn_loss.png
│   │   └── forecast_comparison.png
│   └── results/                  # Numerical results
│       ├── stationarity.csv
│       ├── outliers.csv
│       ├── granger.csv
│       ├── johansen.csv
│       ├── residual_diagnostics.csv
│       ├── forecast_metrics.csv
│       ├── diebold_mariano.csv
│       └── summary.json
├── python/
│   ├── config.py                 # Configuration
│   ├── data_load.py              # Data loading
│   ├── diagnostics.py            # Statistical tests
│   ├── models_classical.py       # ARIMA, VAR, VECM, GARCH
│   ├── models_dl.py              # LSTM, GRU, hybrid
│   └── run_pipeline.py           # Python pipeline
├── R/
│   ├── config.R                  # R configuration
│   ├── data_load.R               # R data loading
│   ├── diagnostics.R             # R statistical tests
│   ├── models_classical.R         # R classical models
│   ├── run_pipeline.R            # R pipeline
│   ├── install_packages.R        # Package installation
│   └── README.md                 # R documentation
├── notebook.ipynb                # Jupyter notebook (main executable)
├── REPORT.md                     # Project report
└── requirements.txt              # Python dependencies
```

B.2 Running the Code

B.2.1 Python Implementation

Option 1: Jupyter Notebook (Recommended)
```bash
jupyter notebook notebook.ipynb
```

Option 2: Python Script
```bash
cd python
python run_pipeline.py
```

B.2.2 R Implementation

```r
# Install packages first
source("R/install_packages.R")

# Run pipeline
source("R/run_pipeline.R")
```

B.3 Dependencies

Python
- numpy, pandas
- yfinance
- statsmodels
- pmdarima
- arch
- tensorflow/keras
- matplotlib, seaborn
- scikit-learn

R
- quantmod
- forecast
- vars
- urca
- rugarch
- tseries
- ggplot2
- lmtest
- nortest

End of Report